

**UNIVERSIDADE DO ESTADO DE MINAS GERAIS
UNIDADE DIVINÓPOLIS**

CURSO DE ENGENHARIA DE COMPUTAÇÃO

ÍCARO GABRIEL DOS SANTOS

RAPHAEL MURO PIMENTEL

**SISTEMA DE RECUPERAÇÃO DE INFORMAÇÃO:
COMPARAÇÃO ENTRE O MODELO BOOLEANO
E O MODELO DE ESPAÇO VETORIAL**

Relatório final apresentado à disciplina SISTEMAS DE RECUPERAÇÃO DE INFORMAÇÃO, como parte das atividades avaliativas, contemplando a implementação, análise experimental e comparação de dois modelos clássicos de Recuperação de Informação: o modelo Booleano e o modelo de Espaço Vetorial.

**DIVINÓPOLIS – MG
2026**

Sumário

1	INTRODUÇÃO E OBJETIVOS	3
1.1	Introdução	3
1.2	Objetivo geral	3
1.3	Objetivos específicos	4
2	METODOLOGIA E IMPLEMENTAÇÃO	5
2.1	Descrição da coleção	5
2.2	Arquitetura geral	5
2.3	Pré-processamento textual	6
2.4	Construção do índice invertido	7
2.5	Modelo Booleano	7
2.6	Modelo de Espaço Vetorial	8
2.7	Interface do sistema	8
3	RESULTADOS	10
3.1	Configuração do experimento	10
3.2	Resultados do modelo Booleano	10
3.3	Resultados do modelo de Espaço Vetorial	11
3.4	Exemplos de consultas e rankings	12
4	DISCUSSÃO: ANÁLISE CRÍTICA, BENEFÍCIOS E LIMITAÇÕES .	13
4.1	Comparação direta dos modelos	13
4.2	Benefícios do modelo Booleano	13
4.3	Limitações do modelo Booleano	14
4.4	Benefícios do modelo vetorial	14
4.5	Limitações do modelo vetorial	14
4.6	Possíveis melhorias	15
5	CONCLUSÃO	16
6	LIÇÕES APRENDIDAS	17
7	REFERÊNCIAS BIBLIOGRÁFICAS	18
	REFERÊNCIAS	19
8	USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA (IAG)	21

8.1	Declaração de uso	21
8.2	Partes do trabalho que receberam apoio de IAG	21
8.3	Prompts utilizados	21
8.4	Responsabilidade intelectual	22
8.5	Limitações do uso de IAG	22
A	PROCEDIMENTO PARA EXECUÇÃO DO SISTEMA	23
A.1	Indexação	23
A.2	Interface gráfica	23
A.3	Exemplos de teste	23
B	RESUMO TÉCNICO DOS ARQUIVOS DO PROJETO	25

1 Introdução e objetivos

1.1 Introdução

A Recuperação de Informação (RI) é a área da Computação dedicada a localizar, organizar e apresentar informações relevantes diante de uma necessidade de informação expressa, normalmente, por meio de uma consulta. Em uma coleção textual, um Sistema de Recuperação de Informação (SRI) precisa transformar documentos e consultas em representações que permitam identificar quais documentos atendem melhor à necessidade do usuário.

Neste trabalho foi desenvolvido e analisado um SRI aplicado a uma coleção de 20 documentos acadêmicos relacionados principalmente à Engenharia de Software, qualidade de software, testes, cobertura de código, refatoração, análise de defeitos e utilização de modelos de linguagem. O sistema utiliza os resumos dos artigos como base para indexação e disponibiliza dois mecanismos de recuperação: o modelo Booleano e o modelo de Espaço Vetorial.

O modelo Booleano trabalha com conjuntos de documentos e operadores lógicos, como AND, OR e NOT. Dessa forma, uma consulta define condições que devem ser satisfeitas para que um documento seja recuperado. O resultado é, portanto, um conjunto de documentos, sem uma ordenação intrínseca por grau de relevância.

Já o modelo de Espaço Vetorial representa documentos e consultas por vetores de pesos. Na implementação desenvolvida, esses pesos são obtidos pelo esquema TF-IDF e a comparação entre documento e consulta é feita pela similaridade do cosseno. Isso permite ordenar os documentos de acordo com um valor numérico de similaridade, fornecendo uma noção de grau de relevância.

A comparação entre os dois modelos é especialmente importante porque evidencia um dos principais compromissos dos sistemas de recuperação: uma estratégia mais restritiva pode apresentar alta precisão e deixar documentos relevantes de fora, enquanto uma estratégia baseada em similaridade pode recuperar mais documentos relevantes, mas também trazer resultados que não são relevantes para a necessidade específica do usuário.

1.2 Objetivo geral

Implementar, executar e avaliar um Sistema de Recuperação de Informação capaz de recuperar documentos de uma coleção acadêmica utilizando os modelos Booleano e de Espaço Vetorial, comparando seus resultados por meio das métricas de precisão e revocação sobre um conjunto de julgamentos de relevância definido pelo grupo.

1.3 Objetivos específicos

- organizar uma coleção de 20 artigos acadêmicos e seus respectivos metadados;
- realizar o pré-processamento textual dos resumos;
- remover palavras consideradas *stopwords* e normalizar os termos;
- construir vocabulários individuais para os documentos;
- construir um índice invertido para permitir a recuperação dos documentos;
- implementar consultas Booleanas com **AND**, **OR**, **NOT** e parênteses;
- implementar o modelo de Espaço Vetorial utilizando TF-IDF e similaridade do cosseno;
- disponibilizar uma interface gráfica para realização das consultas;
- executar cinco consultas fixas de teste;
- definir julgamentos de relevância para as consultas e calcular precisão e revocação;
- comparar criticamente os dois modelos, identificando vantagens, limitações e possibilidades de melhoria.

2 Metodologia e implementação

2.1 Descrição da coleção

A coleção utilizada no experimento é composta por 20 artigos acadêmicos, armazenados em arquivos PDF. Os documentos abordam temas de Engenharia de Software, incluindo testes de software, qualidade, cobertura de código, manutenção, refatoração, análise estática, bugs, testes baseados em modelos e modelos de linguagem.

Para a implementação do SRI, os títulos, autores e resumos foram consolidados no arquivo `Titulos__autores_e_resumos.txt`. A tabela `tabela_documentos.csv` registra o identificador de cada documento, título, autor e quantidade total de termos significativos. O arquivo `registro_controle.txt` registra o maior identificador e o total de termos significativos processados pela coleção, resultando em `<20, 2065>`.

Além dos metadados, o projeto possui 20 arquivos de vocabulário, um para cada documento, e um arquivo de 392 linhas de palavras utilizadas como *stopwords*. O vocabulário registra cada termo e sua frequência no documento, no formato `<termo, frequência>`.

2.2 Arquitetura geral

A solução foi organizada em módulos Python, cada um com uma responsabilidade específica:

- `parser.py`: leitura e separação dos artigos, títulos, autores e resumos;
- `indexador.py`: normalização, tokenização, remoção de *stopwords* e geração de vocabulários;
- `main.py`: execução do processo de indexação e geração dos arquivos auxiliares;
- `recuperacao.py`: carregamento do índice e implementação dos modelos Booleano e Vetorial, além do cálculo de precisão e revocação;
- `interface.py`: interface gráfica para entrada de consultas, seleção do modelo e visualização dos resultados.

A Figura 1 apresenta o fluxo geral do sistema.

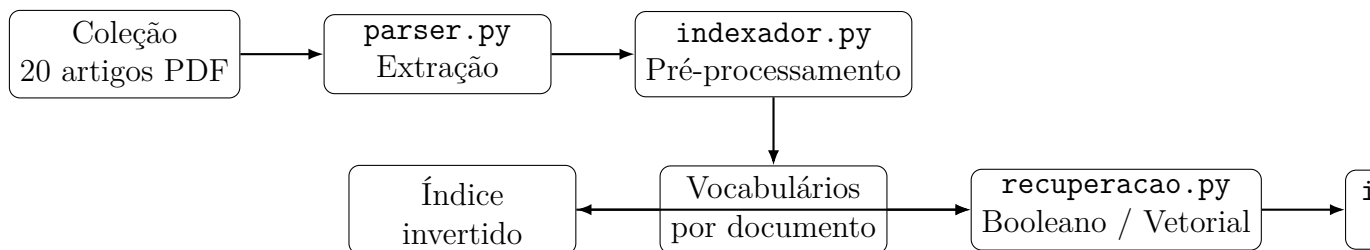


Figura 1 – Arquitetura simplificada do Sistema de Recuperação de Informação.

2.3 Pré-processamento textual

O pré-processamento é fundamental porque a comparação entre documentos e consultas depende da representação dos textos. O código realiza as seguintes etapas:

1. conversão do texto para letras minúsculas;
2. tokenização por expressão regular;
3. normalização Unicode, removendo marcas de acentuação;
4. remoção de *stopwords*;
5. descarte de tokens com apenas um caractere;
6. contagem das ocorrências dos termos restantes.

A normalização é realizada pela função `normalizar_palavra`. Conceitualmente, termos como “Qualidade” e “qualidade” passam a ter a mesma representação. A remoção de acentos também reduz diferenças artificiais entre termos.

O processo de geração do vocabulário pode ser resumido pelo seguinte trecho da implementação:

Listing 2.1 – Geração do vocabulário após o pré-processamento.

```

1 def gerar_vocabulario(resumo, stopwords):
2     tokens = tokenizar(resumo)
3     termos = []
4
5     for token in tokens:
6         token_normalizado = normalizar_palavra(token)
7         if token_normalizado in stopwords:
8             continue
9         if len(token_normalizado) <= 1:
10             continue
11         termos.append(token_normalizado)
  
```

12
13

```
return Counter(termos)
```

2.4 Construção do índice invertido

Depois de gerar os vocabulários individuais, o sistema constrói um índice invertido. Em vez de procurar termo por termo dentro de todos os documentos, o índice associa cada termo ao conjunto de documentos nos quais ele aparece.

Por exemplo, se o termo *cobertura* aparece nos documentos 8, 9, 18 e 19, o índice mantém uma estrutura equivalente a:

cobertura -> {8, 9, 18, 19}.

Essa estrutura é particularmente adequada ao modelo Booleano, pois as operações lógicas podem ser executadas diretamente sobre conjuntos.

2.5 Modelo Booleano

O modelo Booleano implementado aceita os operadores AND, OR e NOT, além de parênteses. A precedência adotada é:

1. NOT;
2. AND;
3. OR.

O analisador foi implementado por meio de um parser recursivo. Para uma consulta como:

teste AND software

sejam D_{teste} o conjunto de documentos contendo “teste” e $D_{software}$ o conjunto contendo “software”. O resultado é:

$$R = D_{teste} \cap D_{software}.$$

Para uma consulta com OR, utiliza-se a união:

$$R = D_a \cup D_b.$$

Para NOT, o código considera o universo como o conjunto de todos os 20 documentos:

$$R = U - D_a.$$

Uma característica importante é que o modelo Booleano não produz um valor de relevância. O resultado é simplesmente um conjunto de documentos, que na interface é apresentado sem *score*.

2.6 Modelo de Espaço Vetorial

No modelo vetorial, cada documento é representado por um vetor de pesos. A implementação utiliza TF-IDF. Para um termo t em um documento d , o peso é calculado por:

$$w_{t,d} = TF_{t,d} \times IDF_t.$$

O IDF é calculado pelo código como:

$$IDF_t = \log \left(\frac{N}{df_t} \right),$$

em que N representa o número total de documentos da coleção e df_t representa a quantidade de documentos que contêm o termo.

A consulta também é transformada em vetor TF-IDF. Depois, a similaridade entre a consulta q e um documento d é calculada pela similaridade do cosseno:

$$\text{sim}(q, d) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|}.$$

O valor varia entre 0 e 1 para os vetores não negativos utilizados neste sistema. Quanto maior o valor, maior a proximidade entre a representação da consulta e a representação do documento.

A implementação ordena os resultados de forma decrescente pelo *score*. Documentos cujo *score* seja exatamente zero não são apresentados.

2.7 Interface do sistema

A interface foi implementada com Tkinter e permite ao usuário:

1. digitar uma consulta;
2. escolher entre “Booleano” e “Espaço Vetorial”;
3. executar a busca;
4. visualizar DocId, título, autor e, no modelo vetorial, o valor de relevância;

5. abrir o PDF do documento recuperado por meio de duplo clique.

Essa organização separa a camada de apresentação da lógica de recuperação, permitindo que os modelos sejam comparados utilizando a mesma coleção e a mesma interface.

3 Resultados

3.1 Configuração do experimento

Foram definidas cinco consultas fixas de teste. Para possibilitar a avaliação, o grupo estabeleceu previamente quais documentos deveriam ser considerados relevantes para cada necessidade de informação. O julgamento foi feito a partir do conteúdo disponível nos títulos e resumos dos documentos do corpus, e não apenas da presença literal dos termos da consulta.

A tabela 1 apresenta as consultas, seus julgamentos de relevância e a quantidade de documentos relevantes considerada como referência.

Tabela 1 – Consultas fixas e julgamentos de relevância.

Consulta	Expressão	Documentos considerados relevantes
Q1	teste AND software	1, 8, 10, 14, 15, 16, 17, 18, 19, 20
Q2	cobertura AND codigo	8, 9, 18, 19
Q3	qualidade AND software	1–20
Q4	llms AND bugs	13, 17
Q5	testes AND automatizados	14, 16, 17, 18, 19, 20

Observação sobre os julgamentos. A consulta Q3 possui julgamento de relevância igual à coleção inteira porque todos os documentos apresentam relação explícita ou contextual com qualidade de software, seja como objetivo principal, seja como critério ou preocupação do trabalho. Esse julgamento deve ser entendido como uma decisão experimental do grupo, e não como uma afirmação de que todos os documentos possuem o mesmo grau de relevância.

3.2 Resultados do modelo Booleano

A tabela 2 mostra os documentos recuperados pelo modelo Booleano e as métricas obtidas.

Tabela 2 – Resultados do modelo Booleano.

Consulta	Documentos recuperados	Relevantes recuperados	Precisão	Revocação	F1
Q1	1, 8, 14, 15, 16, 17, 18	7	1,0000	0,7000	0,823
Q2	8, 9, 18	3	1,0000	0,7500	0,857
Q3	1, 2, 3, 5, 6, 7, 8, 9, 16, 17, 19	11	1,0000	0,5500	0,709
Q4	13	1	1,0000	0,5000	0,666
Q5	19, 20	2	1,0000	0,3333	0,500
Média	–	–	1,0000	0,5667	0,711

O comportamento observado é consistente com a natureza do modelo. Nas cinco consultas, todos os documentos recuperados foram considerados relevantes segundo o julgamento adotado, resultando em precisão média de 1,0000. Em contrapartida, a revocação média foi de 0,5667, indicando que uma parcela dos documentos considerados relevantes não foi recuperada.

Um exemplo claro é Q5. O sistema recuperou apenas os documentos 19 e 20, apesar de o julgamento considerar relevantes também os documentos 14, 16, 17 e 18. Isso ocorre porque a recuperação Booleano depende da presença simultânea dos termos da expressão no vocabulário dos documentos. A estratégia é precisa, porém rígida.

3.3 Resultados do modelo de Espaço Vetorial

No modelo vetorial, a recuperação é ordenada pelos valores de similaridade. Os resultados completos obtidos nas cinco consultas são apresentados a seguir.

Tabela 3 – Resultados do modelo de Espaço Vetorial.

Consulta	Documentos recuperados	Relevantes recuperados	Precisão	Revocação	F1
Q1	19	10	0,5263	1,0000	0,6897
Q2	7	4	0,5714	1,0000	0,7273
Q3	20	20	1,0000	1,0000	1,0000
Q4	3	2	0,6667	1,0000	0,8000
Q5	7	6	0,8571	1,0000	0,9231
Média	–	–	0,7243	1,0000	0,8280

A característica mais marcante do modelo vetorial foi a revocação de 1,0000 em todas as cinco consultas. Isso significa que, segundo os julgamentos adotados, todos os documentos relevantes foram recuperados em cada consulta. Entretanto, essa recuperação mais ampla veio acompanhada de documentos não relevantes, principalmente nas consultas Q1 e Q2.

Na consulta Q1, por exemplo, o modelo recuperou 19 dos 20 documentos. Embora todos os 10 documentos relevantes tenham sido encontrados, nove documentos adicionais também foram apresentados. A precisão, portanto, caiu para 0,5263.

Na consulta Q5, o comportamento foi melhor equilibrado: seis dos seis documentos relevantes foram recuperados, juntamente com apenas um documento adicional. Isso produziu precisão de 0,8571 e F1 de 0,9231.

3.4 Exemplos de consultas e rankings

Para demonstrar a diferença entre os modelos, a consulta Q4 foi particularmente representativa:

11ms AND bugs

No modelo Booleano, apenas o documento 13 foi recuperado. Já o modelo vetorial retornou:

Tabela 4 – Ranking da consulta Q4 no modelo vetorial.

Posição	Documento	Score
1	13	0,2808
2	11	0,1623
3	17	0,0524

O documento 17 foi considerado relevante pelo julgamento humano, embora não tenha sido recuperado pelo Booleano. O modelo vetorial, por outro lado, conseguiu recuperar o documento 17 devido à proximidade textual existente na representação dos documentos, mesmo que os dois termos da consulta não apareçam simultaneamente na forma necessária para satisfazer a conjunção Booleana.

4 Discussão: análise crítica, benefícios e limitações

4.1 Comparação direta dos modelos

A tabela 5 resume os valores médios obtidos.

Tabela 5 – Comparação dos modelos avaliados.

Modelo	Precisão média	Revocação média	F1 médio
Booleano	1,0000	0,5667	0,7114
Espaço Vetorial	0,7243	1,0000	0,8280

Os resultados mostram uma diferença clara. O modelo Booleano apresentou precisão perfeita no conjunto de testes, porém recuperou somente cerca de 56,67% dos documentos considerados relevantes. O modelo vetorial apresentou revocação perfeita, mas precisão média inferior, de 72,43%.

Essa diferença não significa que um modelo seja universalmente melhor que o outro. Ela mostra que os modelos priorizam propriedades diferentes. O Booleano obedece estritamente às condições especificadas pelo usuário. Se a consulta exigir dois termos simultaneamente, um documento que não contenha os dois termos não será retornado, mesmo que seu conteúdo seja claramente relacionado ao assunto.

O modelo vetorial possui comportamento mais flexível. A utilização de pesos TF-IDF e similaridade do cosseno permite que documentos sejam ordenados pela proximidade da representação textual. Essa propriedade é útil quando o usuário não sabe exatamente quais termos aparecem nos documentos ou quando deseja encontrar documentos semanticamente próximos da consulta lexical.

4.2 Benefícios do modelo Booleano

- interpretação simples e previsível;
- controle explícito dos critérios da busca;
- excelente precisão nas consultas avaliadas;
- suporte natural a filtros com NOT;
- implementação eficiente com conjuntos e índice invertido.

4.3 Limitações do modelo Booleano

- não apresenta ranking de relevância;
- é sensível à escolha exata dos termos;
- uma conjunção muito restritiva pode eliminar documentos relevantes;
- não diferencia graus de relevância entre documentos recuperados;
- depende de o usuário formular adequadamente a expressão lógica.

4.4 Benefícios do modelo vetorial

- produz ranking por similaridade;
- recupera documentos mesmo quando a correspondência exata da expressão é inexistente;
- utiliza a frequência dos termos e sua capacidade de discriminação na coleção;
- apresentou maior revocação nos testes realizados;
- fornece um valor quantitativo de similaridade para comparação entre documentos.

4.5 Limitações do modelo vetorial

A implementação atual apresenta uma limitação importante: não existe um limiar de relevância. Todo documento com *score* maior que zero é apresentado. Isso explica parte da quantidade elevada de falsos positivos nas consultas mais gerais.

Outra limitação é que o modelo vetorial implementado utiliza correspondência lexical. Apesar de ser mais flexível que o Booleano, ele não compreende semanticamente que palavras diferentes podem expressar conceitos semelhantes. Por exemplo, termos relacionados a “falhas”, “defeitos” e “bugs” podem representar conceitos próximos em determinado contexto, mas não são automaticamente tratados como sinônimos.

Também se observa que os operadores Booleanos presentes na consulta são ignorados na construção do vetor da consulta. Assim, no modelo vetorial, **AND**, **OR** e **NOT** não alteram a consulta como ocorre no modelo Booleano. Eles são filtrados e somente os termos textuais são utilizados para o cálculo TF-IDF.

4.6 Possíveis melhorias

Com base nos resultados, algumas melhorias podem ser propostas:

1. aplicar *stemming* ou lematização para reduzir variações morfológicas;
2. utilizar uma tokenização mais robusta para termos técnicos e siglas;
3. implementar pesos TF-IDF normalizados de forma mais sofisticada;
4. criar um limiar configurável de similaridade no modelo vetorial;
5. permitir ao usuário escolher quantos resultados deseja visualizar, por exemplo, Top-5 ou Top-10;
6. implementar expansão de consultas e tratamento de sinônimos;
7. avaliar métricas adicionais, como MAP e P@k;
8. ampliar o conjunto de julgamentos de relevância com avaliadores independentes;
9. manter versões separadas do índice para diferentes estratégias de pré-processamento;
10. incorporar busca por título e resumo com pesos diferentes.

5 Conclusão

O trabalho permitiu implementar e analisar um Sistema de Recuperação de Informação completo, partindo da coleção de documentos e do pré-processamento textual até a recuperação e avaliação dos resultados. A solução foi estruturada de forma modular, permitindo compreender a função de cada etapa e comparar dois modelos clássicos de RI sobre a mesma coleção.

O modelo Booleano mostrou-se eficiente quando a necessidade de informação pode ser expressa por condições claras e restritivas. No experimento realizado, alcançou precisão média de 1,0000, porém com revocação média de 0,5667. Isso confirma sua característica de produzir resultados mais controlados, mas potencialmente perder documentos relevantes.

O modelo de Espaço Vetorial apresentou o comportamento complementar. A precisão média foi de 0,7243, enquanto a revocação atingiu 1,0000 nas cinco consultas. A ordenação por similaridade e o uso do TF-IDF permitiram recuperar documentos relevantes que não satisfaziam necessariamente todas as condições lexicais exigidas pelo Booleano. O custo dessa flexibilidade foi a recuperação de documentos adicionais que não foram considerados relevantes nos julgamentos.

Considerando o F1 médio, o modelo vetorial obteve 0,8280 contra 0,7114 do modelo Booleano. Entretanto, a escolha do modelo depende do objetivo da aplicação. Em ambientes nos quais o controle explícito da consulta é prioritário, o Booleano continua sendo adequado. Em cenários de exploração documental, nos quais o usuário deseja descobrir documentos relacionados e ordenar resultados, o modelo vetorial apresenta vantagens.

Por fim, o experimento também demonstrou que métricas como precisão e revocação são essenciais para avaliar sistemas de recuperação. Observar somente a quantidade de documentos retornados poderia levar a conclusões equivocadas. A comparação entre os resultados recuperados e um conjunto explícito de julgamentos de relevância fornece uma avaliação mais objetiva do comportamento do sistema.

6 Lições aprendidas

A realização do trabalho permitiu consolidar conhecimentos técnicos e conceituais importantes. As principais lições aprendidas foram:

- **Pré-processamento influencia diretamente a recuperação:** pequenas decisões sobre tokenização, normalização e *stopwords* modificam o vocabulário e, consequentemente, os resultados das consultas.
- **Índice invertido é fundamental para eficiência:** associar termos aos documentos evita que toda a coleção seja percorrida a cada operação lógica.
- **Representação determina o comportamento do SRI:** o Booleano trabalha com pertinência a conjuntos, enquanto o vetorial trabalha com graus de similaridade.
- **Precisão e revocação são complementares:** melhorar uma delas não implica necessariamente melhorar a outra.
- **Ranking é uma vantagem prática:** o modelo vetorial permite priorizar documentos, característica ausente no resultado Booleano básico.
- **Julgamento de relevância é indispensável:** as métricas dependem de uma referência que indique quais documentos devem ser considerados relevantes.
- **Implementação e teoria precisam ser analisadas em conjunto:** uma fórmula como TF-IDF só produz resultados úteis quando as etapas de pré-processamento e indexação são coerentes com ela.
- **Testes controlados revelam limitações:** as cinco consultas mostraram situações em que cada modelo se comporta melhor ou pior, permitindo propor melhorias concretas.

7 Referências bibliográficas

As referências abaixo correspondem aos documentos que compõem o corpus fornecido para a atividade. Quando informações editoriais completas não estavam presentes nos metadados consolidados do projeto, foi mantida a identificação por autor(es), título e documento do corpus, evitando a invenção de dados bibliográficos.

Referências

- [1] BENITTI, Fabiane Barreto Vavassori; ALBANO, Edson Lucas. **Teste de Software: o que e como é ensinado?** Documento do corpus da atividade.
- [2] MORAES, Edson Andrade de; WERNECK, Vera Maria Benjamim. **Uma Abordagem de Avaliação de Qualidade de Aplicações Web.** Documento do corpus da atividade.
- [3] BRANCO JÚNIOR, Eliseu Castelo; BELCHIOR, Arnaldo Dias. **Um Modelo para Avaliação da Qualidade da Gerência de Projetos de Software.** Documento do corpus da atividade.
- [4] FERREIRA, Bruno Nascimento; ARAKAKI, Júlio. **Aplicação de Métricas de Manutibilidade na Re-fatoração de Softwares.** Documento do corpus da atividade.
- [5] NASCIMENTO, João Teixeira do; SOUSA, Caio Rian R. de; CARVALHO FILHO, Edivar C.; BORGES, Guilherme P.; RABELO, Jacilane H.; BEZERRA, Carla Ilane Moreira. **Do Code Smell ao Código Limpo: Uma Experiência Prática no Ensino de Refatoração para Manutenção de Software.** Documento do corpus da atividade.
- [6] CORDEIRO, Aline Gomes; FREITAS, André Luís Policani. **Priorização de requisitos e avaliação da qualidade de software segundo a percepção dos usuários.** Documento do corpus da atividade.
- [7] DUARTE, Katia Cristina; FALBO, Ricardo de Almeida. **Uma Ontologia de Qualidade de Software.** Documento do corpus da atividade.
- [8] SILVA, José Diego Saraiva da. **Entendendo a relação entre integração contínua e cobertura de código.** Documento do corpus da atividade.
- [9] GOMES, Fladson Thiago Oliveira. **Uma abordagem para análise de cobertura de código em cenários de evolução.** Documento do corpus da atividade.
- [10] LIMA, Yury Alencar et al. **Comparação de ferramentas de análise estática para detecção de defeitos de software usando mutantes.** Documento do corpus da atividade.
- [11] SANTOS, Igor Ramsés Temóteo dos et al. **Um Mapeamento Sistemático sobre Técnicas de Classificação e Categorização de Bugs em Software.** Documento do corpus da atividade.

- [12] MACEDO, Charles Mendes de; DELGADO, Karina Valdivia; WIESE, Igor. **Detecção automática de falhas em software JavaScript: Uma revisão sistemática da literatura.** Documento do corpus da atividade.
- [13] RODRIGUES, Gabriel R.; SANTOS, Paulo C. **A Aplicação de LLMs na Detecção e Correção de Bugs: Uma revisão da literatura.** Documento do corpus da atividade.
- [14] ALMEIDA, Érika Regina Campos de. **Geração automática de casos de testes executáveis a partir de casos de teste abstratos para aplicações web.** Campinas, 2013. Documento do corpus da atividade.
- [15] DIAS NETO, Arilo Claudio; TRAVASSOS, Guilherme Horta. **Seleção de Técnicas de Teste Baseado em Modelos.** Documento do corpus da atividade.
- [16] VICCARI, Leonardo Davi. **Automação de teste de software através de linhas de produtos e testes baseados em modelos.** Porto Alegre, 2009. Documento do corpus da atividade.
- [17] GOMES, Adriano Gebert. **Uso de Modelos de Linguagem de Grande Escala (LLMs) para Automação da Geração de Casos de Testes Unitários em Software.** Porto Alegre, 2026. Documento do corpus da atividade.
- [18] LEITÃO, Nuno Miguel Apolinário. **Geração Automática de Teste Unitário com Cobertura de Código Otimizada Integração no Frama-C.** Covilhã, 2014. Documento do corpus da atividade.
- [19] WITTER, Flávia Ishizaki Brocanella; SOUSA, Julia Carolina da Silva; MOLINA, Mariângela Ferreira Fuentes. **Métodos para garantir a qualidade do software e melhorar a cobertura e eficiência dos testes automatizados.** Documento do corpus da atividade.
- [20] SPIRLANDELI, Cleber; ROLAND, Carlos Eduardo de França. **A utilização de testes automatizados no desenvolvimento de software.** Revista EduFatec, v. 2, n. 2, 2019. Documento do corpus da atividade.

8 Uso de Inteligência Artificial Generativa (IAG)

8.1 Declaração de uso

Ferramentas de Inteligência Artificial Generativa foram utilizadas como instrumentos de apoio à elaboração deste trabalho. O uso foi realizado de maneira declarada e supervisionada pelos integrantes do grupo. A IAG foi tratada como ferramenta auxiliar, não como autora do trabalho.

A responsabilidade intelectual pelo código, pela seleção das consultas, pelos julgamentos de relevância, pela interpretação dos resultados, pelas análises críticas e pela versão final do relatório é dos integrantes do grupo.

8.2 Partes do trabalho que receberam apoio de IAG

O apoio de IAG foi utilizado nas seguintes atividades:

1. organização e revisão da estrutura do relatório;
2. apoio à redação e revisão linguística de trechos explicativos;
3. explicação dos módulos e funções do código Python;
4. apoio na explicação conceitual de índice invertido, modelo Booleano, TF-IDF e similaridade do cosseno;
5. apoio na organização das tabelas de resultados;
6. conferência da interpretação das métricas de precisão, revocação e F1;
7. apoio à formulação e revisão das consultas de teste;
8. sugestão de melhorias e limitações para a discussão crítica;
9. apoio à estruturação deste documento em LaTeX.

8.3 Prompts utilizados

Para manter transparência sobre a utilização de IAG, são registrados abaixo os principais tipos de prompts utilizados no processo:

1. **Análise do projeto:** “Analise todos os arquivos do projeto de SRI, explique a função de cada arquivo e descreva o fluxo de execução do sistema.”
2. **Explicação do código:** “Explique didaticamente como o código implementa pré-processamento, vocabulário, índice invertido, busca booleana e busca vetorial.”
3. **Modelo Booleano:** “Explique como o parser recursivo implementa a precedência NOT > AND > OR e como os conjuntos de documentos são operados.”
4. **Modelo vetorial:** “Explique o cálculo de TF-IDF e da similaridade do cosseno utilizado no código, relacionando as fórmulas às funções implementadas.”
5. **Avaliação:** “Com base nas cinco consultas e nos julgamentos de relevância, calcule precisão e revocação para os dois modelos e compare os resultados.”
6. **Relatório:** “Estruture um relatório acadêmico completo contendo capa, sumário, introdução, objetivos, metodologia, resultados, discussão, conclusão, lições aprendidas, referências e declaração de uso de IAG.”
7. **Revisão:** “Revise o texto do relatório procurando inconsistências entre a descrição dos algoritmos e o comportamento real do código.”

8.4 Responsabilidade intelectual

As sugestões produzidas pela IAG foram avaliadas pelos integrantes antes de serem incorporadas ao trabalho. Resultados experimentais foram obtidos a partir da execução do código do projeto e não foram inventados para adequar o relatório a uma conclusão previamente definida. As decisões de julgamento de relevância e a interpretação final dos resultados pertencem aos integrantes do grupo.

8.5 Limitações do uso de IAG

A IAG pode produzir explicações ou sugestões tecnicamente plausíveis que não correspondam exatamente à implementação. Por esse motivo, a análise final foi baseada na leitura do código e na execução efetiva do sistema. Essa verificação foi especialmente importante para as fórmulas de TF-IDF, para a lógica do parser Booleano, para os resultados das cinco consultas e para o cálculo das métricas.

A Procedimento para execução do sistema

O projeto pode ser executado localmente com Python 3. O diretório deve manter a estrutura original, especialmente a pasta `artigos`, o arquivo `stopwords.txt`, a pasta `vocabularios` e os arquivos Python.

A.1 Indexação

A partir da pasta do projeto, execute:

```
1 python main.py
```

A execução realiza o processamento dos resumos, gera ou atualiza os vocabulários e produz a tabela de documentos e o registro de controle.

A.2 Interface gráfica

Depois da indexação, execute:

```
1 python interface.py
```

Na interface, digite a consulta, selecione o modelo e clique em “Buscar”. No modelo vetorial, o resultado apresenta o *score* de similaridade. No modelo Booleano, o campo de relevância é substituído por “-”.

A.3 Exemplos de teste

As consultas utilizadas no experimento podem ser digitadas da seguinte forma:

```
1 teste AND software
2 cobertura AND codigo
3 qualidade AND software
4 llms AND bugs
5 testes AND automatizados
```

Para testar outros recursos do parser Booleano, também podem ser utilizadas expressões como:

```
1 teste AND (software OR qualidade)
2 software AND NOT javascript
3 (teste OR cobertura) AND software
```

No modelo vetorial, os operadores lógicos são descartados pela implementação e somente os termos textuais da consulta participam do cálculo TF-IDF.

B Resumo técnico dos arquivos do projeto

Tabela 6 – Função dos principais arquivos do projeto.

Arquivo	Função
<code>parser.py</code>	Lê o arquivo de títulos, autores e resumos e separa cada artigo por identificador.
<code>indexador.py</code>	Implementa normalização, tokenização, carregamento de stopwords e geração dos vocabulários.
<code>main.py</code>	Coordena a indexação e gera os arquivos de controle e tabela de documentos.
<code>recuperacao.py</code>	Implementa índice invertido, parser Booleano, TF-IDF, cos-seno, busca e métricas.
<code>interface.py</code>	Fornece a interface gráfica e integra a entrada do usuário com os mecanismos de busca.
<code>stopwords.txt</code>	Lista de palavras removidas durante o pré-processamento.
<code>vocabularios/</code>	Contém um vocabulário por documento, com frequência dos termos.
<code>tabela_documentos.csv</code>	Metadados e total de termos significativos por documento.
<code>registro_controle.txt</code>	Registro de controle da coleção processada.
<code>artigos/</code>	PDFs originais dos 20 documentos da coleção.